



Note:

- During the attendance check a sticker containing a unique code will be put on this exam.
- This code contains a unique number that associates this exam with your registration number.
- This number is printed both next to the code and to the signature field in the attendance check list.

Maschinelles Lernen

Exam: IN2064 / Endterm

Date: Friday 24th February, 2023

Examiner: Prof. Günnemann

Time: 17:00 – 19:00

	P 1	P 2	P 3	P 4	P 5	P 6	P 7	P 8	P 9
I									

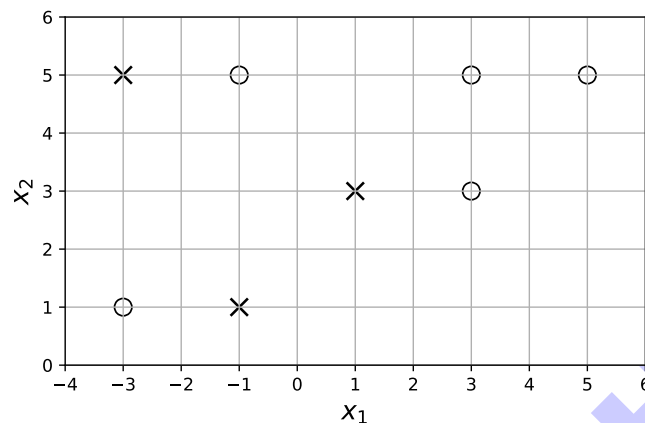
Working instructions

- This exam consists of **16 pages** with a total of **9 problems**.
Please make sure now that you received a complete copy of the exam.
- The total amount of achievable credits in this exam is 36 credits.
- Detaching pages from the exam is prohibited.
- Allowed resources:
 - Two-sided DIN A4 sheet of handwritten notes (a print of digitally handwritten notes is allowed).
- **No other material (e.g. books, cell phones, calculators) is allowed!**
- Physically turn off all electronic devices, put them into your bag and close the bag.
- There is scratch paper at the end of the exam (after problem 9).
- Write your answers only in the provided solution boxes or the scratch paper.
- If you solve a task on the scratch paper, clearly reference it in the main solution box.
- All sheets (including scratch paper) have to be returned at the end.
- **Only use a black or a blue pen (no pencils, red or greens pens!)**
- **For problems that say “Justify your answer” you only get points if you provide a valid explanation.**
- **For problems that say “Derive” you only get points if you provide a valid mathematical derivation.**
- **For problems that say “Prove” you only get points if you provide a valid mathematical proof.**
- If a problem does not say “Justify your answer”, “Derive” or “Prove”, it is sufficient to only provide the correct answer.

Left room from _____ to _____ / Early submission at _____

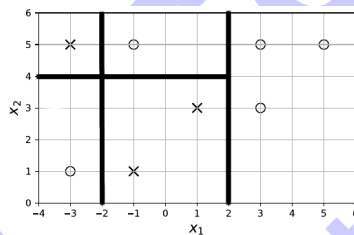
Problem 1 Decision trees (4 credits)

Consider the following two-dimensional classification dataset with the classes “0” (○ marker) and 1 (x marker).

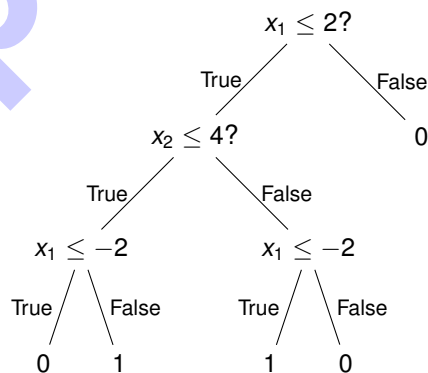


- a) Draw a decision tree of maximum depth 3 that correctly classifies all datapoints. Each decision node must be of the form $x_d \leq c$ with $d \in \{1, 2\}$ and $c \in \mathbb{R}$. Also annotate each edge with “True” or “False” and each leaf node with “0” or “1”

The feature space can be partitioned as follows:

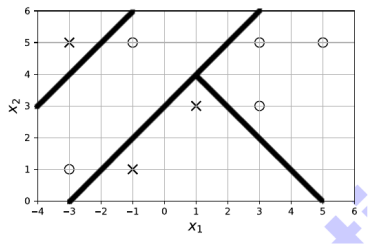


This corresponds to the following decision tree:

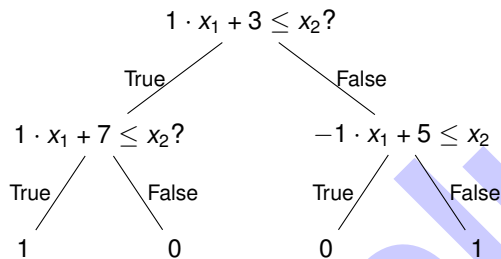


b) We now consider a modified form of decision trees that **only** allows for decision nodes of the form $a \cdot x_1 + b \leq x_2$ with $a, b \in \mathbb{R}$. In particular, nodes of the form $x_1 \leq c$ are **not allowed**. Draw such a decision tree of maximum depth 2 that correctly classifies all datapoints. Also annotate each edge with “True” or “False” and each leaf node with “0” or “1”

The feature space can be partitioned as follows:



This corresponds to the following decision tree:



Problem 2 Probabilistic inference (3 credits)

Consider an infinite number of barns arranged on a regular grid $\mathbb{Z}^2$, with $\mathbb{Z}$ being the set of all integers. An owl starts exploring the world at an unknown location $\mathbf{x}^{(0)} \in \mathbb{Z}^2$. Each day, it moves in one of four directions, according to the following distribution:

$$\begin{aligned} \Pr \left[\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \begin{bmatrix} -1 \\ 0 \end{bmatrix} \mid \mathbf{x}^{(t)} \right] &= \frac{2}{8} & \Pr \left[\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \begin{bmatrix} +1 \\ 0 \end{bmatrix} \mid \mathbf{x}^{(t)} \right] &= \frac{2}{8} \\ \Pr \left[\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} \mid \mathbf{x}^{(t)} \right] &= \frac{3}{8} & \Pr \left[\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \begin{bmatrix} 0 \\ +1 \end{bmatrix} \mid \mathbf{x}^{(t)} \right] &= \frac{1}{8} \end{aligned}$$

- 0 ☐ a) After two days, you find the owl sleeping in $\mathbf{x}^{(2)} = \begin{bmatrix} 6 & 8 \end{bmatrix}^\top$. List all possible starting locations, i.e. all $\mathbf{s} \in \mathbb{Z}^2$
1 ☐ such that $\Pr \left[\mathbf{x}^{(2)} = \begin{bmatrix} 6 & 8 \end{bmatrix}^\top \mid \mathbf{x}^{(0)} = \mathbf{s} \right] > 0$.

(6, 8), (7, 9), (7, 7), (5, 9), (5, 7), (8, 8), (4, 8), (6, 10), (6, 6).

- 0 ☐ b) **Derive** the maximum likelihood estimate for the starting location $\mathbf{x}^{(0)}$, i.e. $\operatorname{argmax}_{\mathbf{s}} \Pr \left[\mathbf{x}^{(2)} = \begin{bmatrix} 6 & 8 \end{bmatrix}^\top \mid \mathbf{x}^{(0)} = \mathbf{s} \right]$.
1 ☐
2 ☐

- (6, 8): $2 \cdot \frac{2}{8} \cdot \frac{2}{8} + 2 \cdot \frac{1}{8} \cdot \frac{3}{8} = \frac{14}{64}$
- (5, 9), (7, 9): $2 \cdot \frac{2}{8} \cdot \frac{3}{8} = \frac{12}{64}$
- (5, 7), (7, 7): $2 \cdot \frac{2}{8} \cdot \frac{1}{8} = \frac{4}{64}$
- (4, 8), (8, 8): $\frac{2}{8} \cdot \frac{2}{8} = \frac{4}{64}$
- (6, 10): $\frac{3}{8} \cdot \frac{3}{8} = \frac{9}{64}$
- (6, 6): $\frac{1}{8} \cdot \frac{1}{8} = \frac{1}{64}$

Starting location $\begin{bmatrix} 6 & 8 \end{bmatrix}^\top$ is the most likely.

Note that when we start to the left (6, 8) of (6, 8), we need to go to the right, which makes the starting location less likely. For instance, the likelihood of (6, 6) is smaller than that of (6, 10).

Problem 3 Linear regression (5 credits)

We want to perform regularized linear regression (without bias) on a dataset with N samples $\mathbf{x}_i \in \mathbb{R}^D$ with corresponding targets y_i (represented compactly as $\mathbf{X} \in \mathbb{R}^{N \times D}$ and $\mathbf{y} \in \mathbb{R}^N$). You assume that your targets are normal distributed, i.e.,

$$p(y_i | \mathbf{x}_i, \mathbf{w}) = \mathcal{N}(\mathbf{x}_i^\top \mathbf{w}, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2} (y_i - \mathbf{x}_i^\top \mathbf{w})^2\right). \quad (3.1)$$

To add regularization, you choose a Laplace prior on the parameters $\mathbf{w} \in \mathbb{R}^D$, i.e.,

$$p(\mathbf{w}) = \frac{1}{2\lambda} \prod_{i=1}^D \exp\left(-\frac{|w_i|}{\lambda}\right). \quad (3.2)$$

with hyperparameter $\lambda > 0$.

a) **Derive** the negative logarithm of the posterior distribution, i.e., $-\log p(\mathbf{w} | \mathbf{X}, \mathbf{y})$ (up to some normalization constant).

$$p(\mathbf{w} | \mathbf{X}, \mathbf{y}) = \frac{p(\mathbf{y} | \mathbf{X}, \mathbf{w}) p(\mathbf{w})}{p(\mathbf{y} | \mathbf{X})} \quad (3.3)$$

$$\propto p(\mathbf{y} | \mathbf{X}, \mathbf{w}) p(\mathbf{w}) \quad (3.4)$$

$$= \prod_{i=1}^N \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2} (y_i - \mathbf{x}_i^\top \mathbf{w})^2\right) \frac{1}{2\lambda} \prod_{i=1}^D \exp\left(-\frac{|w_i|}{\lambda}\right) \quad (3.5)$$

$$\propto \prod_{i=1}^N \exp\left(-\frac{1}{2} (y_i - \mathbf{x}_i^\top \mathbf{w})^2\right) \prod_{i=1}^D \exp\left(-\frac{|w_i|}{\lambda}\right) \quad (3.6)$$

$$\log p(\mathbf{w} | \mathbf{X}, \mathbf{y}) = \frac{1}{2} \sum_{i=1}^N (y_i - \mathbf{x}_i^\top \mathbf{w})^2 - \frac{1}{\lambda} \sum_{i=1}^D |w_i| \quad (3.7)$$

b) What is the advantage/difference of having such a Laplace prior over a Gaussian prior? **Justify your answer!**

The Laplace distribution is more fat-tailed than a Gaussian due to the lower exponent, i.e., it penalizes larger values less while small values are penalized to a larger amount due to the sharp decrease around 0. As an optimal point estimate, the Laplace distribution results in the median while the Gaussian results in the mean. Thus, sparsity is encouraged.

Problem 4 Optimization (6 credits)

Below, you are given two different functions and asked to **prove** convexity.

a) Prove that the subsequent function is convex in $\mathbf{x} \in \mathbb{R}_{>0}^d$, i.e., over the set of vectors solely consisting of positive entries $x_i > 0$ for all $i = 1, \dots, d$:

$$f(\mathbf{x}) = \sum_{i=1}^d x_i \log x_i$$

Hint: Remember that a matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ is positive semi-definite, if $\forall \mathbf{z} \in \mathbb{R}^d : \mathbf{z}^\top \mathbf{A} \mathbf{z} \geq 0$

$$\nabla^2 f(\mathbf{x})_{ij} = \frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} = \begin{cases} 0 & \text{if } i \neq j \\ \frac{1}{x_i} & \text{otherwise} \end{cases} \quad (4.1)$$

Therefore, for an arbitrary $\mathbf{z} \in \mathbb{R}^n$

$$\mathbf{z}^\top \nabla^2 f(\mathbf{x}) \mathbf{z} = \sum_{i=1}^n z_i \left(\sum_{j=1}^n \nabla^2 f(\mathbf{x})_{ij} z_j \right) \quad (4.2)$$

$$= \sum_{i=1}^n \sum_{j=1}^n z_i \nabla^2 f(\mathbf{x})_{ij} z_j \quad (4.3)$$

$$= \sum_{i=1}^n \frac{1}{x_i} z_i^2 \quad (4.4)$$

$$\geq 0 \quad (4.5)$$

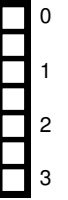
Line 4.4 follows from Equation 4.1 and the last inequality from $x_i > 0$.
Because the Hessian of $f(\mathbf{x})$ is pos. semi-definitive, $f(\mathbf{x})$ is convex.

b) Let $f_1 : \mathbb{R}^d \rightarrow \mathbb{R}$ and $f_2 : \mathbb{R}^d \rightarrow \mathbb{R}$ be two convex functions. **Prove** that

$$h(\mathbf{x}) = \max\{f_1(\mathbf{x}), f_2(\mathbf{x})\}$$

is a convex function.

Note: For this, you are not allowed to use any convexity rules from the lecture without proving them.



Assume $x, y \in \mathbb{R}^d$, $\lambda \in [0, 1]$. We show that the definition of a convex function holds.

$$h(\lambda x + (1 - \lambda)y) = \max\{f_1(\lambda x + (1 - \lambda)y), f_2(\lambda x + (1 - \lambda)y)\} \quad (4.6)$$

$$\leq \max\{\lambda f_1(x) + (1 - \lambda)f_1(y), \lambda f_2(x) + (1 - \lambda)f_2(y)\} \quad (4.7)$$

$$\leq \max\{\lambda f_1(x) + \lambda f_1(y)\} + \max\{(1 - \lambda)f_1(y), (1 - \lambda)f_2(y)\} \quad (4.8)$$

$$= \lambda \max\{f_1(x), f_2(x)\} + (1 - \lambda) \max\{f_1(y), f_2(y)\} \quad (4.9)$$

$$= \lambda h(x) + (1 - \lambda)h(y) \quad (4.10)$$

Line 4.7 follows from the convexity of f_1 and f_2 . Line 4.9 holds, because λ is non-negative. This proves $h(x)$ is convex.

Problem 5 Deep Learning (5 credits)

The following code snippets all contain **exactly one error**. Your task is to spot the mistakes and explain how to fix it. **Justify your answer!**

We omitted variable initializations to avoid clutter. Assume that all variables were appropriately initialized.

0 ☐ a) Given an input $\mathbf{x} \in \mathbb{R}^d$, the subsequent class implements the ReLU layer $f(x) = \max(0, x)$ and the corresponding backward pass.

1 ☐ import numpy as np

2 ☐

```
class ReLU:
    def forward(self, inputs):
        self.cache = inputs
        out = np.maximum(inputs, 0)
        return out
    def backward(self, d_out):
        inputs = self.cache
        d_inputs = d_out * (inputs < 0)
        return d_inputs
```

```
relu = ReLU()
z = relu.forward(x)
d_x = relu.backward(1.0)
```

The backward pass through the ReLU layer is wrong (1p).

$$\frac{\partial \text{ReLU}(x)}{\partial x} = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases} \quad (5.1)$$

The fix (1p) should thus be:

```
d_inputs = d_out * (inputs > 0)
```

☐	0
☐	1
☐	2
☐	3

b) We have trained a model to perform multiclass classification over c classes on a dataset $\mathbf{X} \in \mathbb{R}^{n \times d}$ and one-hot encoded targets $\mathbf{y} \in \{0, 1\}^{n \times c}$ with $\sum_{j=1}^c y_{ij} = 1 \quad \forall i \in [1, \dots, n]$. The model is defined as: $\text{outputs} = \text{ReLU}(\mathbf{x} @ \mathbf{w}_1 + \mathbf{b}_1) @ \mathbf{w}_2 + \mathbf{b}_2$. The model was trained to minimize the Cross Entropy between the one-hot encoded target and the prediction. Now, we want to obtain the normalized class probabilities as well as the predicted class.

```
import torch

model.eval()
outputs = model.forward(x)
classprobs = torch.sigmoid(outputs)
y_hat = torch.argmax(classprobs, axis=1)
for i, (cp, yh) in enumerate(zip(classprobs, y_hat)):
    print(f"predicted class {yh} for sample {i} with probability {cp[yh]}")
```

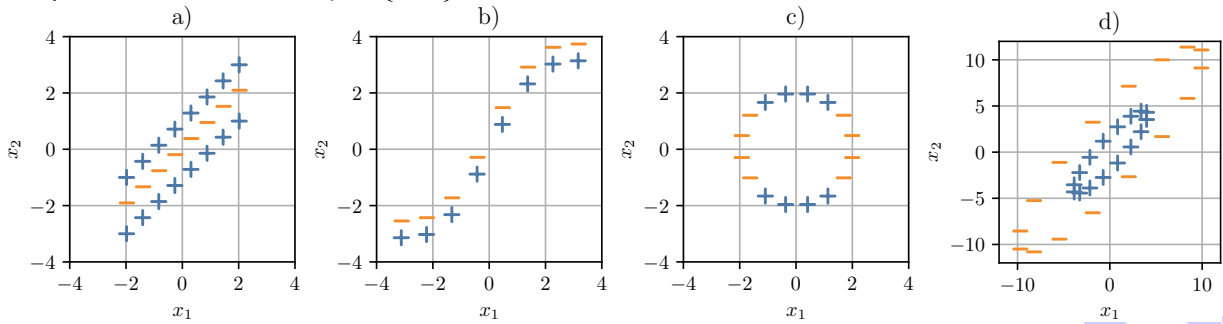
The sigmoid over a multiclass output will not give normalized class probabilities (1p). Instead, the softmax should be taken over the output (1p for softmax, 1p for axis kw):

```
classprobs = torch.softmax(outputs, dim=-1)
```

Problem 6 Linear classification (4 credits)

0	
1	
2	
3	
4	

We want to perform binary classification on four different datasets, $t \in \{a, b, c, d\}$, each consisting of N_t samples $\mathbf{x}_i \in \mathbb{R}^2$ with labels $y_i \in \{-, +\}$:



You already came up with transformations $\phi_1, \dots, \phi_4$ that transform the respective datasets such that they are linearly separable:

$$\phi_1(\mathbf{x}) = \hat{x}_1 \hat{x}_2 \quad (6.1)$$

$$\hat{\mathbf{x}} = \mathbf{x} \begin{bmatrix} \cos \frac{\pi}{4} & -\sin \frac{\pi}{4} \\ \sin \frac{\pi}{4} & \cos \frac{\pi}{4} \end{bmatrix} \quad (6.2)$$

$$\phi_2(\mathbf{x}) = x_2 - \sin(x_1) - x_1 \quad (6.2)$$

$$\phi_3(\mathbf{x}) = \left\| \begin{bmatrix} \frac{x_1}{2} \\ x_2 - x_1 \end{bmatrix} \right\|_2 \quad (6.3)$$

$$\phi_4(\mathbf{x}) = |x_1 - x_2| \quad (6.4)$$

Unfortunately, you forgot which transform belongs to which dataset. Assign the transformations $\phi_1, \phi_2, \phi_3, \phi_4$ to the datasets a, b, c, d such that the transformed datasets are linearly separable. **Justify your answer!**

We give four points, half a point for each matching and half a point for each justification. The correct pairs are:

- (a, 4): subtracting x_1 from x_2 maps each point to either -1, 0, or 1. The absolute value then maps -1 to 1. So, afterward the - are at 0 and all plus are mapped to 1.
- (b, 2): Similar to a, by subtracting $\sin(x_1)$ we get a similar result to a but with - being the upper line. Thus, subtracting x_1 also maps the data points either to 0 (-) or 1 (+) depending on their class.
- (c, 1): Here we first need to rotate the dataset by 45° (thus the rotation matrix), then the data points are clearly distinguished by the sign of the product of both coordinates.
- (d, 3): Here the classes differ by radius in an ellipsoid. The ellipsoid is stretched in x direction and correlated in y direction. The transformations in (3) reverse the transformations such that the euclidean norm (L2) norm distinguishes both circles.

Problem 7 Support Vector Machines and Kernels (4 credits)

You are given a dataset with N datapoints $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$ representing the class of datapoint i . We use the augmentation trick $\mathbf{x} \mapsto \tilde{\mathbf{x}} = (\mathbf{x}, 1)$ to turn the affine decision function of an SVM classifier $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ (with explicit bias term) into a linear function $\tilde{h}(\mathbf{x}) = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$ with $\tilde{\mathbf{w}} = (\mathbf{w}, b) \in \mathbb{R}^{d+1}$.

Now, we want to solve the adapted (maximum-margin) optimization problem

$$\begin{aligned} \min_{\tilde{\mathbf{w}}} \quad & \frac{1}{2} \tilde{\mathbf{w}}^\top \tilde{\mathbf{w}} \\ \text{subject to} \quad & y_i \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i - 1 \geq 0 \quad i = 1, \dots, N \end{aligned}$$

a) What is the Lagrangian function $L(\tilde{\mathbf{w}}, \alpha)$ associated to the above problem, with $\alpha_i \geq 0$ corresponding to the Lagrangian multipliers.

0
1

$$L(\tilde{\mathbf{w}}, \alpha) = \frac{1}{2} \tilde{\mathbf{w}}^\top \tilde{\mathbf{w}} - \sum_{i=1}^N \alpha_i (y_i \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i - 1) \quad (7.1)$$

b) **Derive** the corresponding dual function $g(\alpha)$. It suffices to simplify $g(\alpha)$ such that it does not contain any minimization or maximization term.

0
1
2
3

Hint: The Lagrangian function $L(\tilde{\mathbf{w}}, \alpha)$ is convex in $\tilde{\mathbf{w}}$.

The dual function is defined as $g(\alpha) = \min_{\tilde{\mathbf{w}}} L(\tilde{\mathbf{w}}, \alpha)$. Because $L(\tilde{\mathbf{w}}, \alpha)$ is convex, it suffices to set its derivative w.r.t. $\tilde{\mathbf{w}}$ to zero.

$$\nabla_{\tilde{\mathbf{w}}} L(\tilde{\mathbf{w}}, \alpha) = \tilde{\mathbf{w}} - \sum_{i=1}^N \alpha_i y_i \tilde{\mathbf{x}}_i = 0 \quad \rightarrow \quad \tilde{\mathbf{w}}^* = \sum_{i=1}^N \alpha_i y_i \tilde{\mathbf{x}}_i \quad (7.2)$$

Plugging the derived $\tilde{\mathbf{w}}^*$ back into $g(\alpha)$ gives the solution

$$g(\alpha) = L(\tilde{\mathbf{w}}^*, \alpha) \quad (7.3)$$

$$= \frac{1}{2} \left\| \sum_{i=1}^N \alpha_i y_i \tilde{\mathbf{x}}_i \right\|_2^2 - \sum_{i=1}^N \alpha_i \left(y_i \left(\sum_{j=1}^N \alpha_j y_j \tilde{\mathbf{x}}_j \right)^\top \tilde{\mathbf{x}}_i - 1 \right) \quad (7.4)$$

Up to here should be enough.

Problem 8 PCA (3 credits)

0 ☐
1 ☐
2 ☐
3 ☐

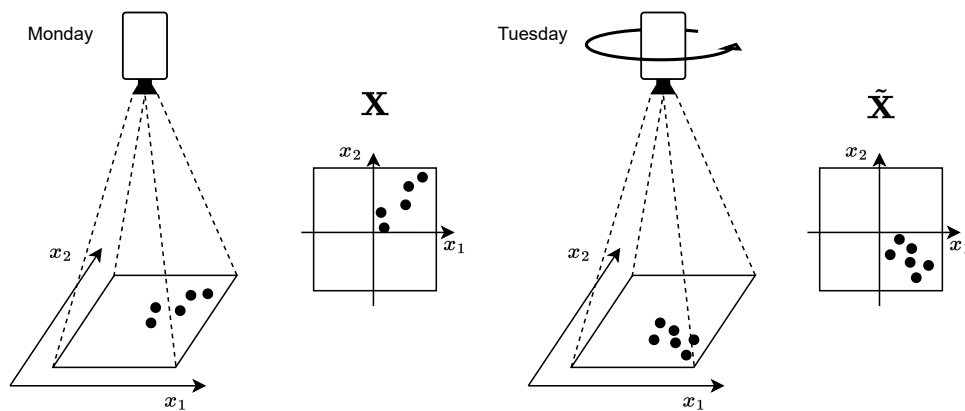
On Monday, you experimented with growing bacteria and took a photo of the result. You recorded the positions of bacteria as illustrated below. Each position is a two-dimensional coordinate, with the origin in the middle of the camera's frame. The positions are saved in a data matrix $\mathbf{X} \in \mathbb{R}^{N \times 2}$. On Tuesday, you repeated the experiment but did not set up the camera at the same angle. Tuesday's measurements are denoted with $\tilde{\mathbf{X}} \in \mathbb{R}^{M \times 2}$.

Since you assume the positions will follow the **same distribution** every day, you want to rotate the data recorded on Tuesday to **match the direction and shape** of the data from Monday. Unfortunately, the only data processing technique you know is PCA. Fortunately, this is enough to solve this problem. **Propose a solution and justify your answer.**

You have a function $\text{PCA}(\mathbf{D})$ at your disposal, which takes data matrix $\mathbf{D} \in \mathbb{R}^{a \times b}$ and returns $\mathbf{\Gamma} \in \mathbb{R}^{b \times b}$ corresponding to the principal components, and $\mathbf{\Lambda} \in \mathbb{R}^b$ corresponding to the eigenvalues. You also know the commands for basic matrix manipulation: addition, subtraction, multiplication and transpose.

Assume that PCA always gives you the desired eigenvectors, that is, ignore the potential sign flips in $\mathbf{\Gamma}$.

Note: Figure below is just for illustration purposes. The angle and the values N and M are not given.



New data is assumed to be rotated by some random rotation matrix $\mathbf{R}$. If the data wasn't rotated it would have the same principal components.

Thus, the covariance of the original data can be factorized as $\mathbf{\Gamma}\mathbf{\Lambda}\mathbf{\Gamma}^T$ while the covariance of the rotated data can be factorized as $\mathbf{R}^T\mathbf{\Gamma}\mathbf{\Lambda}\mathbf{\Gamma}^T\mathbf{R}$.

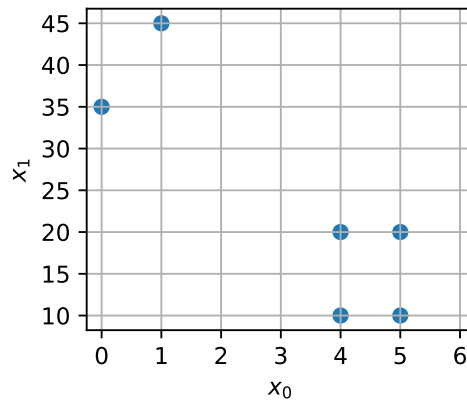
We only need to find $\mathbf{R}$ and then rotate. To do that perform the following:

1. $\mathbf{\Gamma}_1 = \text{PCA}(\mathbf{X})$
2. $\mathbf{\Gamma}_2 = \text{PCA}(\tilde{\mathbf{X}})$
3. $\mathbf{R}^T = \mathbf{\Gamma}_2\mathbf{\Gamma}_1^T = \mathbf{R}^T\mathbf{\Gamma}_1\mathbf{\Gamma}_1^T$
4. $\tilde{\mathbf{X}} \leftarrow \tilde{\mathbf{X}}\mathbf{R}^T$

Important, we have to multiply with $\mathbf{R}^T$ to revert the rotation. It's not necessary to center the data.

Problem 9 Clustering (2 credits)

You are given the following two-dimensional dataset $\mathbf{X} \in \mathbb{R}^{n \times d}$:

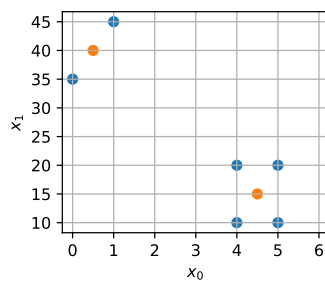


a) What are the globally optimal cluster centers μ that minimize the k-means objective.

$$J(\mathbf{X}, \mathbf{Z}, \mu) = \sum_{i=1}^N \sum_{k=1}^K \mathbf{z}_{ik} \|\mathbf{x}_i - \mu_k\|_2^2 \quad (9.1)$$

0
1

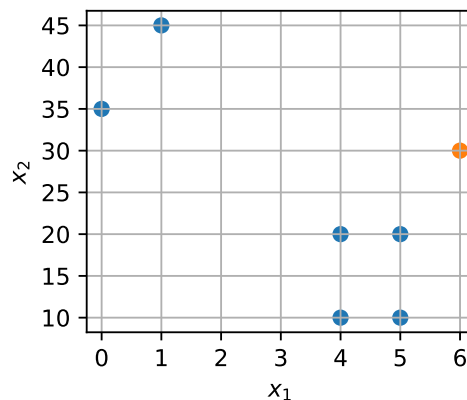
where $\mathbf{Z}$ is a mutually exclusive assignment of datapoints to the closest cluster center μ . **Justify your answer!**



(0.5, 40) and (4.5, 15)

b) Now assume you want to infer the corresponding cluster for a new datapoint without updating the cluster centers μ . To what cluster center in μ does the following point correspond to? **Justify your answer!**

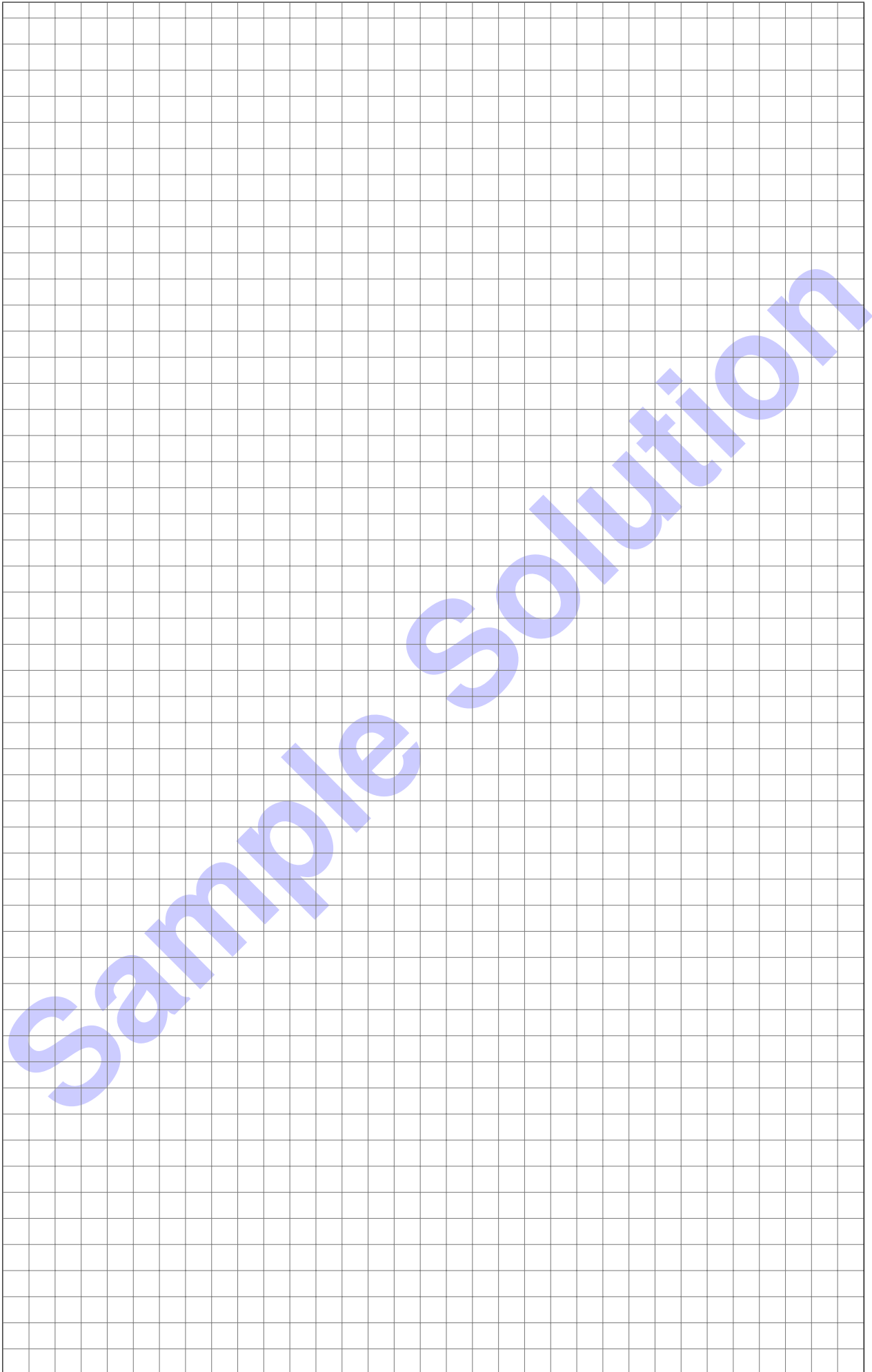
0
1



The new point belongs to the cluster with mean (0.5, 40), i.e., the cluster consisting of example (0, 35) and (1, 45).

Additional space for solutions—clearly mark the (sub)problem your answers are related to and strike out invalid solutions.

A large grid of graph paper, consisting of 30 columns and 30 rows of small squares. A diagonal watermark with the text "Sample Solution" in a light blue, sans-serif font is overlaid across the grid, running from the bottom-left towards the top-right.



Sample Solution